

# Python pour le trading — Document 7/8

La qualité, le test et l'outillage

## 1. Les tests unitaires avec pytest

Un **test unitaire** vérifie automatiquement qu'un petit morceau de code se comporte comme prévu. Plutôt que de tester à la main après chaque modification, on écrit des tests qui s'exécutent en une commande. pytest est l'outil standard en Python.

```
# Le code à tester (dans ordres.py)
def valeur_notionnelle(quantite, prix):
    return quantite * prix

# Le test (dans test_ordres.py)
def test_valeur_notionnelle():
    assert valeur_notionnelle(5, 5234.50) == 26172.50    # cas normal
    assert valeur_notionnelle(0, 5234.50) == 0           # cas limite

def test_valeur_avec_quantite_negative():
    assert valeur_notionnelle(-2, 100) == -200           # comportement attendu
```

Un test est une fonction dont le nom commence par `test_`. À l'intérieur, `assert` vérifie qu'une condition est vraie : si elle l'est, le test passe ; sinon, il échoue et pytest le signale. On lance tous les tests d'un coup avec la commande `pytest`. L'intérêt : après chaque modification, on relance les tests et on sait immédiatement si quelque chose est cassé. On teste le cas normal, mais aussi les **cas limites** (zéro, valeurs négatives), là où se cachent les bugs.

**Pourquoi ça compte.** Sur un système de trading, un bug introduit par une modification peut être très coûteux. Une batterie de tests détecte instantanément une régression avant qu'elle n'atteigne la production. C'est un filet de sécurité que tout employeur sérieux attend.

## 2. Le mocking : tester sans dépendances réelles

Comment tester une fonction qui appelle une API de courtier, sans réellement passer d'ordre ni dépendre du réseau ? Avec un **mock** : un faux objet qui imite le vrai, et qu'on contrôle. Cela rend les tests rapides, fiables et sans effet de bord.

```
from unittest.mock import Mock

def envoyer_ordre(courtier, ordre):
    reponse = courtier.passer(ordre)    # appelle le vrai courtier
    return reponse["statut"]

def test_envoyer_ordre():
    faux_courtier = Mock()              # un faux courtier
    faux_courtier.passer.return_value = {"statut": "execute"} # réponse simulée

    resultat = envoyer_ordre(faux_courtier, {"symbole": "ES"})
    assert resultat == "execute"
    faux_courtier.passer.assert_called_once() # vérifie qu'on l'a bien appelé
```

Le Mock remplace le vrai courtier par un faux qu'on programme : ici, on lui dit de renvoyer {"statut": "execute"} quand on appelle passer. Le test vérifie alors le comportement de envoyer\_ordre sans toucher à un vrai courtier — aucun ordre réel, aucune dépendance au réseau. On peut même vérifier que la fonction a bien appelé le courtier, avec assert\_called\_once. C'est indispensable pour tester du code qui interagit avec l'extérieur.

**Pourquoi ça compte.** On ne peut pas passer de vrais ordres pour tester. Le mocking permet de vérifier toute la logique de trading sans risque ni dépendance réseau, ce qui rend les tests reproductibles et sûrs — une compétence attendue dans tout code financier.

### 3. Le logging : garder une trace

Le **logging** (journalisation) consiste à enregistrer ce que fait le programme, avec des niveaux d'importance. C'est radicalement supérieur aux print : on peut filtrer par gravité, horodater, et écrire dans un fichier. Dans un système temps réel, c'est souvent le seul moyen de comprendre après coup ce qui s'est passé.

```
import logging

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

log.debug("Détail technique, ignoré en production")
log.info("Ordre ES envoyé, quantité 5")
log.warning("Latence élevée détectée")
log.error("Échec de connexion au courtier")
log.critical("Perte de tous les flux de marché")
```

Question : ici, il manque clairement d'explications avec l'Exemple. Toutes les méthodes/attribus/etc utilisés avec logging, les paramètres, etc.

Le logging propose des **niveaux** de gravité croissante : DEBUG (détails), INFO (déroulement normal), WARNING (anomalie non bloquante), ERROR (échec), CRITICAL (urgence). En réglant le niveau (ici INFO), on choisit ce qu'on garde : en production, on ignore les DEBUG pour ne pas noyer l'information. Chaque message est horodaté automatiquement. Contrairement aux print, on peut rediriger ces journaux vers un fichier et les analyser plus tard.

**Pourquoi ça compte.** Quand un système tourne en continu et qu'un problème survient à 3h du matin, les journaux sont la seule façon de reconstituer ce qui s'est passé. Un logging soigné, par niveaux, est vital pour surveiller et diagnostiquer un système de trading — exactement la surveillance demandée dans l'offre.

### 4. Le débogage

Quand un bug résiste, le **débogueur** permet d'arrêter le programme à un endroit précis et d'inspecter l'état des variables, pas à pas. Python intègre pdb.

```
def calculer_pnl(ordres):
    total = 0
    for ordre in ordres:
```

```

        breakpoint()          # le programme s'arrête ICI
        total += ordre.valeur # on peut inspecter 'ordre' et 'total'
    return total

```

La fonction `breakpoint()` met le programme en pause à cet endroit et ouvre une invite où l'on peut afficher les variables (taper le nom d'une variable), exécuter **ligne par ligne**, et comprendre exactement ce qui se passe. C'est bien plus puissant que de parsemer le code de `print` : on explore l'état réel du programme au moment du bug. Les commandes courantes sont `n` (ligne suivante), `c` (continuer), `p` variable (afficher).

***Pourquoi ça compte.** Un débogueur fait gagner un temps considérable face à un bug subtil : au lieu de deviner, on observe directement l'état du programme. Sur du code de trading complexe, c'est un outil de diagnostic essentiel.*

## 5. Le profilage de performance

Quand un programme est trop lent, le **profilage** identifie **où** il passe son temps, au lieu de deviner. Inutile d'optimiser au hasard : on mesure d'abord. Python fournit `cProfile` et `timeit`.

```

import cProfile

# Profiler : où le temps est-il dépensé ?
cProfile.run("analyser_historique(donnees)")
# -> liste les fonctions par temps cumulé : on voit le goulot d'étranglement

# timeit : mesurer précisément un petit bout de code
import timeit
duree = timeit.timeit("sum(range(1000))", number=10000)

```

Question : cest pas clair ici. De plus, ce nest pas un exemple en lien avec le trading. L'exemple devrait montrer des sorties dd [cprofil.run](#) etc.

`cProfile.run` exécute le code et rapporte, fonction par fonction, le temps passé : on repère immédiatement le **goulot d'étranglement**, c'est-à-dire la partie lente qui mérite d'être optimisée. `timeit` mesure précisément un petit fragment en le répétant plusieurs fois. La règle : **mesurer avant d'optimiser**. Souvent, 90 % du temps est passé dans 10 % du code ; le profilage révèle ces 10 %, pour concentrer l'effort là où il compte.

***Pourquoi ça compte.** Optimiser à l'aveugle fait perdre du temps et complique le code inutilement. Le profilage indique précisément quoi accélérer — crucial quand la latence compte, comme dans un système de trading. C'est le prélude au document 8 sur la performance.*

## 6. Style et structure du code

Un code de production doit être **lisible et cohérent**, car il est relu et modifié par plusieurs personnes. Deux conventions comptent : la norme de style **PEP 8**, et l'organisation en modules.

```

# Un "linter" comme ruff signale automatiquement les écarts de style

```

```
# ruff check mon_projet/

# Organisation en modules (fichiers) et packages (dossiers)
# ordres.py          -> la logique des ordres
# marche.py          -> la connexion au marché
# test_ordres.py     -> les tests
```

La **PEP 8** est la convention de style officielle de Python (nommage, espacement, longueur des lignes). Un **linter** comme ruff la vérifie automatiquement et signale les écarts. Organiser le code en **modules** (fichiers thématiques) et **packages** (dossiers) le rend navigable : chaque responsabilité a sa place. Un code propre et bien rangé est plus facile à comprendre, à tester et à faire évoluer — ce que tout employeur valorise.

***Pourquoi ça compte.** Dans une équipe, du code lisible et cohérent réduit les erreurs et accélère le travail de tous. Respecter PEP 8 et structurer son projet signale un développeur professionnel, au-delà du simple « ça marche ».*

## Récapitulatif

1. **pytest** : des tests automatiques (fonctions test\_ avec assert) ; couvrir les cas normaux et limites.
2. **Mocking** : remplacer une dépendance réelle (courtier, API) par un faux contrôlé, pour tester sans risque.
3. **Logging** : journaliser par niveaux (DEBUG à CRITICAL) plutôt que des print ; indispensable au suivi temps réel.
4. **Débogage** : breakpoint() pour arrêter et inspecter l'état du programme pas à pas.
5. **Profilage** : cProfile/timeit pour mesurer où le temps est passé ; mesurer avant d'optimiser.
6. **Style** : PEP 8 et linters (ruff), organisation en modules et packages.

## Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

### Sur les tests et le mocking

**Q1.** Qu'est-ce qu'un test unitaire et comment l'écrit-on avec pytest ?

**Réponse.** Une vérification automatique qu'un morceau de code se comporte comme prévu. Avec pytest, on écrit une fonction dont le nom commence par test\_ et on y place des assert qui vérifient des conditions.

**Q2.** Pourquoi tester aussi les cas limites ?

**Réponse.** Parce que c'est là que se cachent souvent les bugs : valeurs nulles, négatives, vides, extrêmes. Tester seulement le cas normal laisse passer ces erreurs.

**Q3.** Qu'est-ce qu'un mock et pourquoi l'utiliser ?

**Réponse.** Un faux objet qui imite un vrai (courtier, API) et qu'on contrôle. Il permet de tester la logique sans dépendre du réseau ni provoquer d'effet réel (comme passer un vrai ordre).

## Sur le logging et le débogage

**Q4.** Pourquoi préférer le logging aux print ?

**Réponse.** Le logging propose des niveaux de gravité, horodate les messages, peut écrire dans un fichier et filtrer ce qu'on garde. Les print n'offrent rien de tout cela et sont inadaptés à un système en production.

**Q5.** Cite les niveaux de logging du moins au plus grave.

**Réponse.** DEBUG, INFO, WARNING, ERROR, CRITICAL.

**Q6.** Qu'apporte un débogueur (breakpoint) par rapport aux print ?

**Réponse.** Il arrête le programme à un endroit précis et permet d'inspecter l'état réel des variables, d'avancer ligne par ligne. On observe au lieu de deviner.

## Sur le profilage et le style

**Q7.** Que fait le profilage et quelle est sa règle d'or ?

**Réponse.** Il identifie où le programme passe son temps (le goulot d'étranglement). Règle d'or : mesurer avant d'optimiser, plutôt que d'optimiser au hasard.

**Q8.** Quelle est la différence entre cProfile et timeit ?

**Réponse.** cProfile analyse tout un programme et rapporte le temps par fonction. timeit mesure précisément un petit fragment de code en le répétant plusieurs fois.

**Q9.** Qu'est-ce que la PEP 8 et un linter ?

**Réponse.** La PEP 8 est la convention de style officielle de Python (nommage, espacement...). Un linter comme ruff vérifie automatiquement le respect de ces règles et signale les écarts.

*Document suivant (8/8) : la performance — pourquoi Python peut être lent, la vectorisation comme solution, le multiprocessing pour le calcul, et les accélérateurs numba/Cython.*